

Static analysis and 'safety' of Contracts, P2900 vs. P2680/P3285

Abstract

This paper discusses two things; first, the part that the author finds quite alarming, that various people seem to think that the proposed P2900 Contracts are an effective addition to C++ to make the language or rather the programs written with it 'safer'.

Second, the part that the author finds equally alarming, that the proposed P2900 Contracts provide good-enough support for static analysis based on contract annotations, which is highly related to the first part. It was indeed suggested in the St. Louis meeting that P2900 has enough support for static analysis, and the shortcomings that have been attempted to be discussed were responded to with a rather interesting statement, roughly along the lines of "static analyzers already do useful things with asserts, but if an analyzer *can't prove that an assert has no side effects, such an assert is simply ignored*".

That response is alarming to the hilt. It seemed to have been made as an argument that suggests that the support P2900 lends to static analysis is sufficient, and the emphasized part was intended to suggest that there's no problem with using the P2900 Contracts for static analysis, and that those contracts support static analysis fine, and that static analysis tools can use those contracts just fine.

What is suggested there, in the emphasized part of the paraphrased quote, as a non-problem, is a huge problem. It is **THE** problem.

What EWG, and WG21, need to do is to make sure the Contracts approach proposed in [P2680](#) and [P3285](#) is included in the first incarnation of the Contracts facility that we ship. We must change direction, and we need to do that before we ship anything in this space. We have to do that to have a language facility that is useful for the whole C++ community, and a language facility that effectively and meaningfully improves the safety of C++ programs.

The proposed P2900 Contracts don't do that, and can't. The ones proposed in P2680/P3285 can.

Ignored until proven idempotent

For illustrative purposes, let's consider the following declarations:

```
bool pred(int x);  
int func_a() post(r: pred(r));  
void func_b(int x) pre(pred(x));
```

and the first example of their use is

```
func_b(func_a());
```

Now, the question is, is that code snippet correct?

Can you, the reader of the code, know whether that snippet is correct? Can a static analysis tool tell you that it is correct, or that it doesn't seem to be correct?

The answer is "no" on all accounts. Both you the reader, and an analysis tool, can **guess** that it might be correct, because a postcondition assertion that has exactly the same condition as a subsequent precondition assertion does has been encountered right before the call that calls a function that requires that precondition.

What if `pred()` has a side effect? Well, you can't then tell whether the postcondition and the precondition connect, because you can't tell whether they end up having the same value, because a side effect might cause them not to.

What if `pred()` encounters Undefined Behavior? Well, you can't then tell whether the postcondition and the precondition connect, because you can't tell whether they end up having the same value, because UB might cause them not to.

What if `pred()` doesn't encounter Undefined Behavior, but something in it is optimized away, and produces a result that you didn't expect? Well, in case no UB is encountered, and no side effects occur, the postcondition and the precondition probably connect. But is the program correct? Are the postcondition and the precondition doing what you expect them to do? Maybe not.

So, for our code reader, and our static analysis tool, those post/pre don't tell us much. For better or worse, they give us a hint that we can use to guess that the program might be correct. But that's all they give us. We can't **prove** that, we can't reason about that, without reading a whole lot of other code.

So, if you tell a static analyzer to find **proofs** of correctness or incorrectness of your code, for this particular example it will, at most, say "I can't". If you ask it to find guesses where there are indications of code being correct, it might say that this code has some. But as suggested, an analyzer can't **prove** anything about this code, so it'll by and large just ignore those post/pre, at least for the analysis that attempts to **prove** something.

At this point, let's take a short side-step to the world that P2680/P3285 give us. To do that, we need to change our predicate slightly, and I'm using a context-sensitive keyword instead of an attribute:

```
bool pred(int x) conveyor;  
int func_a() post(r: pred(r));  
void func_b(int x) pre(pred(x));
```

and the example of their use is

```
func_b(func_a());
```

Can you the reader of the code tell whether that code is correct? Yes, you can. The postcondition connects to the precondition. Can a static analyzer prove that? Yes, it can. The predicate is **guaranteed** not to have side-effects or UB. Pay particular attention to the fact that we do **not** need to look at any definitions to come to this **proof**. It can be constructed based on declarations, and code calling functions so declared.

Burden of proof

So, with P2900 Contracts, there are various cases where an attempt to construct a proof simply fails, it just can't be done.

No visible definition

If the definition of `pred` is in another translation unit, and we are not talking about an inline function in a module interface here, we are talking about a case where the definition just isn't visible, we get the following entertaining results:

- For P2900, game over. Neither a human reader nor a static analyzer can construct any proof, either for or against correctness.
- For P2680/P3285, the proof can be constructed as ever, no problem. The postcondition connects to a precondition. We know that. There is a language guarantee of a lack of side-effects and UB. Nothing changed.

Let's look at a slight twist, with slightly different building blocks:

```
class X
{
    virtual bool pred(int x);
public:
    int func_a() post(r: pred(r));
    void func_b(int x) pre(pred(x));
};
```

and the example of their use is

```
void do_stuff(X& x) {
    x.func_b(x.func_a());
}
```

For P2900, it's still game over. No proofs can be constructed, without finding the final overrider of `X::pred`, and looking at its definition.

For P2680/P3285, we do the same tweak as before,

```
class X
{
    virtual bool pred(int x) conveyor;
public:
    int func_a() post(r: pred(r));
    void func_b(int x) pre(pred(x));
};
```

and we have no problem constructing a proof. We can still construct that proof without knowing what the call resolves to, and without looking at the definition of that final overrider.

Definition visible, the cost of proof

First of all, static analyzers available for C++ are fantastic. If you know which one to procure, and have the budget for it. For example, analyzers like Perforce's Helix QAC and Polyspace advertise being able to prove that your code doesn't run into Undefined Behavior. In some cases. And with varying effort/cost, which is configurable; you can tell such an analyzer how hard it will try to construct such a proof.

See, that proof asks the questions, "does this code invoke a side-effect?", and "does this code invoke Undefined Behavior, perhaps as a result of such a side-effect?". To some of us, those questions seem familiar. They are the same question as "does this code *halt*?".

So, for all cases and in general, that proof can't be done. But it is, of course, not quite so bleak. The tools apply symbolic evaluation, do flow analysis, and when so configured, effectively symbolically fuzz your code. The

employ various analyses, some simpler, some more complex, going all the way to a brute-force approach if so configured.

So, depending on the complexity and the amount of code you have in your predicates, that proof-attempt may be quite heavy. And it will be quite different for different invocations of the same predicate, and for different predicates. Sometimes it'll be just impossible, and the analyzer will have to give up and tell you it can't compute the proof.

Consider, for example,

```
class X
{
    virtual bool pred(int x);
public:
    int func_x() post(r: pred(r));
    void func_a(int x) pre(pred(x));
    void func_b(int x) pre(pred(x));
};
```

and a use

```
void do_stuff(X& x) {
    int a = x.func_x();
    x.func_a(a);
    x.func_a(a + 10);
    x.func_b(a);
    x.func_b(a + 10);
}
```

An attempt to prove that that postcondition connects to the preconditions can't necessarily be done just once for the predicate. It may need to be re-proven for the calls with different arguments, taking into account the possible effects of side-effects in the predicate in between.

And again, for P2680/P3285, all we need is a conveyor function as a predicate,

```
class X
{
    virtual bool pred(int x) conveyor;
public:
    int func_x() post(r: pred(r));
    void func_a(int x) pre(pred(x));
    void func_b(int x) pre(pred(x));
};
```

and we do not need to run or re-run any analysis of the predicate. We know that it didn't have side effects or UB. It doesn't have them for any argument values, whereas without that knowledge in hand, the analysis may need to be re-done.

And yes, I know that side-effects in `func_x`, `func_a` or `func_b` can wreck that analysis. But the difference is that for P2900, the *predicate itself* may need to be analyzed multiple times, whereas for P2680/P3285, it doesn't need to be analyzed at all.

There is of course the difference that the compiler will need to do a semantic check for the implementation of `X::pred`. But the compiler will do that **once**. And that result can be reused by multiple analyses by a static analyzer. And multiple runs. All that is required is the knowledge that the code compiled, i.e. followed the rules for 'strict' contracts and conveyor functions.

What the cost and feasibility difference means

Considering the much lighter weight of static analysis that is based on P2680/P3285, that analysis is feasible to do during your build. Or on the fly in your IDE. The analysis never requires looking into predicate definitions, not once, let alone multiple times per run. It can be done looking at calls in a single TU, and looking at the declarations of what's being called.

In practice, a static analysis based on P2900 can't be done near as often. It's much heavier, has false positives and false negatives, and sometimes doesn't produce a useful result. You're not going to enable such analyses in an IDE if you run into problematic cases, and neither will anyone else.

The elephant in the room: safety

When talking about things like memory-safety, look at what modern memory-safe programming languages do: they make memory-unsafe constructs ill-formed, whenever they can. They employ run-time checks when they *have to*, due to halting-problem challenges for things like bounds-checking, but whenever they can, they just reject memory-unsafe constructs. Or make things so that memory-unsafe operations cannot occur (you can't have a dangling reference in a properly garbage-collected language).

Many people in WG21 already think Contracts is a partial solution to safety problems, including memory-safety problems. Apparently the same expectation is held by some people outside WG21.

To avoid causing a huge disappointment, we should ship a Contracts facility that's amenable to finding problems when software is built, not just when it's run. And we should do so effectively, not with a facility whose viability for static analysis depends on far too many things that could go wrong.

It's not just about static analysis, either

The 'strict' contracts proposed in P2680/P3285 aren't just better for static analysis. They are better for runtime-checking as well. When side-effects and UB are not in the picture, it's going to be far easier to write checks that actually do what the programmer intended, without the caveats, pitfalls, and traps of side-effects and UB.

And again, for the millionth time: this doesn't mean that the facilities proposed in P2900 shouldn't exist, and shouldn't be standardized

Since the false argument keeps getting repeated, I'm going to try to dispel it the umpteenth time: this doesn't mean we shouldn't have 'relaxed' contracts. They are fine for some uses, and there are cases where you may not be able to use a 'strict' contract.

However, if we wish to avoid a fiasco of cosmic proportions utterly failing to deliver something actually effective to our users in a reasonable timeframe, we can't just ship the 'relaxed' contracts and pretend we can feasibly ship the 'strict' ones later. They will need to come in the same package, or if that can't be done, the 'strict' ones will need to come first. Otherwise we are looking at a technical disaster and a non-technical disaster.

P2900 can't and won't meet its goals

There's a particularly intriguing realization that occurred to me when working on this problem space. Given P2900, how do we actually increase safety and correctness of C++ programs? You know, considering that we don't have any language help and support for knowing that our *contracts* are correct. Which, one would think, would be a rather important step in knowing that the rest of our programs are correct.

P2680/P3285 give language support for avoiding all sorts of problems in the contracts themselves. P2900 gives none.

So, presumably the answer must be what it is for C++ code in general, since P2900 contracts are C++ code, nothing new, the P2900 contracts have all the things normal C++ code has. And that's suggested as a benefit. So, that means we will run our usual static analyzers on the code in those contracts, and we'll run our usual test approaches on them.

The first problem is that now there's more code to analyze and test, **and** there's new interactions with that additional code and the code it's going to runtime-check. So the overall complexity of the correctness-checking task increases, and it increases in new ways.

The second problem is that our existing practices Do Not Work. They may work for some shops, and for some particular users, but for the user community at large, they Do Not Work. Thousands and thousands of CVEs prove otherwise.

So, now we would be applying the approaches known not to work on *more code*, and need to take into account increased complexity of that code, and *new interactions* in it, interactions of the runtime-checking code and the code that's being checked.

That is logically infeasible. That's logically impossible. We need a different approach. And P2680/P3285 provide us with that. The approach proposed there is better for everybody, including the staunch proponents of P2900, including ones that oppose spending time at this point on P2680/P3285. Everybody, including those two somewhat overlapping audiences, gets a facility that is better for **both** run-time checking **and** static analysis.

And maybe, just maybe, we can talk about Contracts in C++ being an effective safety measure without failing miserably in the field of practice, and having our **users** suffer the various consequences thereof.

Brief rumination about the opposition to P2680/P3285

It's of course not an exact science why SG21 and SG23 members have thought that P2680/P3285 shouldn't be further worked on during the C++26 cycle. But one of the major arguments against that seems to be a thought that it's unnecessary to provide special UB/side-effect protection for contract conditions, when a more general protection thereof in the language will then half-automatically benefit contracts as well.

There's two reasons why that argument is faulty:

- It seems likely that the more general protection is an opt-in, so it might be enabled, it might not be. Nevertheless, what is necessary for contracts is that they are UB-free and side-effect-free regardless of whether the rest of the language is. You can apply contracts on code that's built without any UB protection on. You can apply contracts on code that's built with some sort of UB protection on. In both of those cases, the contracts should be UB-free and side-effect-free.
- It doesn't seem likely that any language profile will provide for the side-effect-free part. If a precondition or a contract assert ends up accidentally modifying a function parameter, that is not a borrowing violation. If a contract assert ends up modifying local variables in a function definition, that is not a borrowing violation.

That is why contracts should have those special protections, regardless of whether the rest of the language does. And the other argument in favor of having those protections especially for contracts still applies: contracts are special, because they are correctness annotations. Correctness annotations are less likely to be correct themselves if all the caveats and bug farms of all the rest of the language are present completely unmitigated.